Draft-Strategic Data Masking & Classification Design Document

Document Control

Document Status

Date	Version	Status
24-Feb-2026	v.0	

Document Authors

Name	Role
Naresh Kala	Platform Team Member
Koustubh Kasalkar	Offshore Lead
Syed Fayaz	Onshore Lead
Pavan Jangam	Platform Team Member

Endorsers and Approvers

Name	Role	Purpose	Version	Date	Status

Key Revision History

Version	Date	Description	Author(s)	Status
0.1	24/02/2026	First Draft	Naresh Kala	Draft

Summary

The architecture ensures that classification metadata extracted from Alation flows through a governed dbt pipeline into the Snowflake Governance Database, where it drives automated tag application and masking policy enforcement across all data product tables.

Objectives

- Automate ingestion of classification metadata from Alation
- Eliminate manual classification entry
- Standardize tagging and masking policies
- Enable Auditable governance

Current State (As-Is)

The following components have been confirmed as existing and available for this initiative

- Manual entries in CLASSIFICATION_DATA
- dbt macro applies tags
- Governance Database .
- Masking policies applied via tags

Target State (To-Be)

The new system will remove manual work and make the process automatic, consistent, and easy to manage.

What will change:

- No more manual entries
  Classification data will come automatically from Alation using Snowflake Data Share.
- One central table
  All classification details will be stored in one place (CLASSIFICATION_DATA).
- Automatic processing using dbt
  dbt will read, clean, and prepare the data.
- Automatic tagging and masking
  dbt macros will:

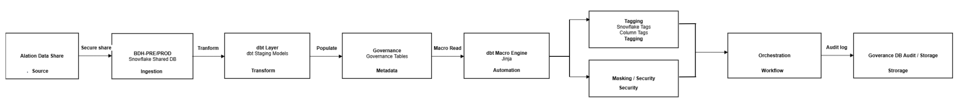
Apply tags to columns

Apply masking policies
No manual steps needed.

- Runs automatically on schedule
  Airflow will run the full process (data tagging masking) daily or as needed.
- Only changes are processed
  System will update only new or changed data (faster and efficient).
- Proper access control
  Only authorized users can manage tags and masking.
- Testing included
  Checks will ensure tagging and masking are working correctly.

Architecture Overview

Alation Snowflake DB BDH-Alation snowflake Shared DB dbt Staging Governance Tables dbt Macro Tags Masking Policies Orchestration Governance DB



This architecture shows how classification data flows from Alation to Snowflake and is used to automatically apply tagging and masking.
Step-by-step Flow

- **Alation Data Share (Source)**
  - Alation provides classification data using Snowflake Data Share
  - This is the starting point (source of truth)
- **Snowflake Shared Database**
  - The shared data is available in Snowflake as a read-only database
  - No data movement or ingestion required
- **dbt Staging Layer**
  - dbt reads the data from the shared database
  - Cleans and standardizes the data
- **Governance Tables**
  - Cleaned data is stored in governance tables like CLASSIFICATION_DATA
  - This acts as the central table for all classifications
- **dbt Macro Engine**
  - dbt macros read the classification data
  - Automatically generate SQL to apply tags and masking
- **Tagging (Snowflake Tags)**
  - Tags are applied to columns (like PII, Confidential)
  - Helps identify sensitive data
- **Masking / Security**
  - Based on tags, masking policies are applied
  - Sensitive data is hidden for unauthorized users
- **Orchestration (Airflow)**
  - Airflow runs the full process on a schedule
  - Ensures everything runs automatically
- **Audit & Storage**
  - All actions (tagging, masking) are logged
  - Stored in governance tables for tracking and auditing

Core Architecture Flow

Layer	Component	Responsibility
Source	Alation snowflake Shared DB	Source of classification metadata (tags, sensitivity, column assignments)
Staging	dbt Staging Models	Extract, validate and standardize Alation share data
Classification	dbt Classification Models	Apply business rules; write to CLASSIFICATION_DATA table
Tagging	dbt Macros (apply_tags)	Execute ALTER TABLE to apply Snowflake tags from classification data
Masking	dbt Macros (apply_masking)	Assign masking policies to tagged columns in Snowflake
Orchestration	Airflow (Astro)	Schedule and sequence the end-to-end pipeline
Governance DB	Snowflake – GOVERNANCE_DB	Target database hosting CLASSIFICATION, TAGGING, MASKING, LOGGING schemas

Data Classification Requirements

Alation Source Metadata

The following fields are extracted from the Alation Snowflake Data Share as source inputs for classification, tagging, and masking. All source references must be validated against the approved STM document before staging development commences.

Source Field	Description	Validation Rule
classification_tag	Classification label applied in Alation	Must match allowed list: PII, PCI, CONFIDENTIAL, PUBLIC, SENSITIVITY_LEVEL
object_type	Type of Snowflake object (TABLE /COLUMN)	Must be TABLE or COLUMN
name	Schema containing the object	Must exist in GOVERNANCE_DB
name	Table name the tag applies to	Must resolve to an existing Snowflake table
name	Column name (null for table-level tags)	Must exist in the referenced table if object_type = COLUMN
ts_updated	Timestamp of classification assignment	Must be a valid timestamp; used for incremental load logic

Data Product Scope

A complete data product table inventory must be agreed with the Data Governance team and documented in the confluence page . The table below represents the classification category framework.

Classification	Sensitivity	Masking Policy	Example Fields
Highly Confidential	High	SENSITIVE_DATA_MASK_STRING	SSN, Credit Card, Government IDs

Confidential	Medium	SENSITIVE_DATA_MASK_STRING/NUMBER	Salary, loan amount, credit score
Private	Low	None	Customer segment, age range
Public	None	None	Product name, status flags
Unclassified	High	SENSITIVE_DATA_MASK_STRING	SSN, Credit Card, Government IDs

Source-to-Target Mapping (STM)

STM Overview

The Source-to-Target Mapping document defines the complete column-level lineage from the Alation Data Share through to the CLASSIFICATION_DATA table in GOVERNANCE_DB. The STM is the governing reference for all dbt model development and must be version-controlled alongside the dbt project.

Target table Structure — CLASSIFICATION_DATA(TBD)

Target Column	**Source Field**	**Transformation**	**Data Type**	**Nullable**
CLASSIFICATION_ID	Derived	HASH(object_schema||object_table||object_column||tag_name)	VARCHAR(64)	No
OBJECT_SCHEMA	object_schema	UPPER(TRIM(object_schema))	VARCHAR(255)	No
OBJECT_TABLE	object_table	UPPER(TRIM(object_table))	VARCHAR(255)	No
OBJECT_COLUMN	object_column	UPPER(TRIM(object_column))	VARCHAR(255)	Yes
OBJECT_TYPE	object_type	UPPER(object_type)	VARCHAR(10)	No
CLASSIFICATION_TYPE	tag_name	UPPER(TRIM(tag_name))	VARCHAR(50)	No
SENSITIVITY_LEVEL	tag_value	UPPER(TRIM(tag_value))	VARCHAR(20)	No
MASKING_POLICY_NAME	Derived	CASE classification_type WHEN PII THEN MASK_PII_HIGH ...	VARCHAR(100)	Yes
ASSIGNED_BY	assigned_by	Direct copy	VARCHAR(255)	Yes
ASSIGNED_DATE	assigned_date	CAST(assigned_date AS TIMESTAMP_NTZ)	TIMESTAMP_NTZ	Yes
RECORD_CREATED_AT	System	CURRENT_TIMESTAMP()	TIMESTAMP_NTZ	No
RECORD_UPDATED_AT	System	CURRENT_TIMESTAMP() on merge	TIMESTAMP_NTZ	No
IS_ACTIVE	Derived	TRUE unless superseded by newer record	BOOLEAN	No

Source table structure

```
WITH temp_table AS (
  SELECT
    am.object_id,--ALATION_SET_MEMBER
    am.object_type,
    am.catalog_set_ids,
    cm.catalog_set_property_id      AS CM_SET_ID,
    cm.catalog_set_title,
    cm.members_object_type          AS object_type,
    rc.column_id,
    rc.name                         AS column_name,
    rc.title                        AS column_title,
    rc.table_id,
    rt.name                         AS table_name,
    rc.schema_id,
    rs.name                         AS schema_name,
    rs.db_catalog_name              AS db_name,
    rc.is_primary_key,
    rc.data_type,
    cm.business_key,
    cm.security_classification,
    --cm.privacy_classification, /* confirmed by shruti */
    /* Convert ARRAY  STRING once */
    ARRAY_TO_STRING(cm.classification_tag, ',') AS classification_tag_str,
    cm.ts_updated
  FROM (
    SELECT
      object_uuid,
      object_id,
      object_type_id,
      object_type,
      f.value::string AS catalog_set_ids
    FROM ALATION__ANALYTICS__SHARE.PUBLIC.ALATION_SET_MEMBER,
         LATERAL FLATTEN(input => catalog_set_ids) f
    WHERE object_type IN ('attribute')
  ) am
  JOIN ALATION__ANALYTICS__SHARE.PUBLIC.CATALOG_SET_MEMBERSHIP cm
    ON am.catalog_set_ids = cm.CATALOG_SET_PROPERTY_ID
  JOIN ALATION__ANALYTICS__SHARE.PUBLIC.RDBMS_COLUMNS rc
    ON am.object_id = rc.column_id
  JOIN ALATION__ANALYTICS__SHARE.PUBLIC.RDBMS_TABLES rt
    ON rc.table_id = rt.table_id
  JOIN ALATION__ANALYTICS__SHARE.PUBLIC.RDBMS_SCHEMAS rs
    ON rt.schema_id = rs.schema_id
  WHERE cm.deleted = FALSE)
```

```sql
SELECT DISTINCT
   db_name        AS DB_NAME,
   schema_name    AS SCHEMA_NAME,
   table_name     AS TABLE_NAME,
   column_name    AS COLUMN_NAME,
   COALESCE(REGEXP_REPLACE(security_classification, '[\{\}""]', ''), 'Unclassified') AS SECURITY_CLASSIFICATION_CODE,

   -- Flag 1: Privacy Identifier (PII)
   CASE
      WHEN UPPER(classification_tag_str) LIKE '%PII%' THEN 'Y'
      ELSE 'N'
   END AS PRIVACY_IDENTIFIER_FLAG,
   -- Flag 2: PCI DSS Compliance
   CASE
      WHEN UPPER(classification_tag_str) LIKE '%PCI DSS%' THEN 'Y'
      ELSE 'N'
   END AS PCI_DSS_COMPLIANCE_IDENTIFIER_FLAG,
   -- Flag 3: Government Identifier
   CASE
      WHEN UPPER(classification_tag_str) LIKE '%GOVERNMENT IDENTIFIER%' THEN 'Y'
      ELSE 'N'
   END AS GOVERNMENT_IDENTIFIER_FLAG,
   -- Flag 4: Sensitive Personal Identifier
   CASE
      WHEN UPPER(classification_tag_str) LIKE '%SENSITIVE PERSONAL%' THEN 'Y'
      ELSE 'N'
   END AS SENSITIVE_PERSONAL_IDENTIFIER_FLAG,
   -- Flag 5: Health Identifier (PHI)
   CASE
      WHEN UPPER(classification_tag_str) LIKE '%PHI%' THEN 'Y'
      ELSE 'N'
   END AS HEALTH_IDENTIFIER_FLAG,
   -- Flag 6: Credit Identifier
   CASE
      WHEN UPPER(classification_tag_str) LIKE '%CREDIT%' THEN 'Y'
      ELSE 'N'
   END AS CREDIT_IDENTIFIER_FLAG,
   -- Flag 7: Credentials Identifier
   CASE
      WHEN UPPER(classification_tag_str) LIKE '%CREDENTIALS%' THEN 'Y'
      ELSE 'N'
   END AS CREDENTIALS_IDENTIFIER_FLAG,

   COALESCE(REGEXP_REPLACE(business_key, '[\{\}""]', ''), FALSE)   AS IS_BUSINESS_KEY,
   COALESCE(is_primary_key, FALSE) AS IS_PRIMARY_KEY,
   data_type                  AS DATA_TYPE,
   catalog_set_title            AS CATALOG_SET_TITLE,
   ts_updated                  AS LAST_UPDATED
FROM temp_table
ORDER BY db_name, schema_name, table_name, column_name;
```

Field	Source Table	Purpose
object_id	public.alation_set_member	Unique identifier
object_type	public.alation_set_member	Object type (attribute/table)
catalog_set_ids	public.alation_set_member	Catalog set associations
column_id	public.rdbms_columns	Column identifier
name	public.rdbms_columns	Column name
table_id	public.rdbms_columns	Table identifier
name	public.rdbms_tables	Table name
schema_id	public.rdbms_schemas	Schema identifier
name	public.rdbms_schemas	Schema name
db_catalog_name	public.rdbms_schemas	database Name
is_primary_key	public.rdbms_columns	Primary key flag
security_classification	public.catalog_set_membership	Classification level
catalog_set_title	public.catalog_set_membership	Catalog set name
ts_updated	public.catalog_set_membership	Update timestamp
ds_id	public.catalog_set_membership	Data source ID (PRE-PROD/PROD)

Merge Key Strategy

Primary merge key for incremental loads: (OBJECT_SCHEMA, OBJECT_TABLE, OBJECT_COLUMN, CLASSIFICATION_TYPE). This composite key uniquely identifies each classification assignment at the column level. For table-level tags where OBJECT_COLUMN is null, OBJECT_COLUMN should be stored as the literal string '__TABLE__' to enable the merge key to function correctly.

Incremental Processing

Strategy

- Use record_updated_at timestamp
- Hash-based change detection

Handling Changes

- New records Insert
- Updated Update
- Removed Optional delete/flag

Masking Policy Name Mapping

Classification Code	Masking Policy	Notes
Unclassified	SENSITIVE_DATA_MASK_STRING	Treated as Highly Confidential (safe default)
Highly Confidential	SENSITIVE_DATA_MASK_STRING	Full masking for non-HC roles
Highly Confidential (keys)	HASH_KEY (SHA-256)	Hashing for BK/PK/FK
Confidential	SENSITIVE_DATA_MASK_STRING	Partial masking
Confidential (keys)	HASH_KEY (SHA-256)	Hashing for BK/PK/FK
Private	Plaintext	Plaintext
Public	Plaintext	Plaintext

Added:

- Policy Assignment Rules (4 decision points)
- Data Type Specific Policies (5 data types):
  - VARCHAR/TEXT/STRING SENSITIVE_DATA_MASK_STRING '***MASKED***'
  - NUMBER/INT/DECIMAL SENSITIVE_DATA_MASK_NUMBER -1
  - DATE SENSITIVE_DATA_MASK_DATE '0001-01-01'
  - TIMESTAMP SENSITIVE_DATA_MASK_TIMESTAMP Date only
  - BOOLEAN SENSITIVE_DATA_MASK_BOOLEAN True/False

dbt Architecture Design

Project Structure

A new dbt project will be created targeting GOVERNANCE_DB. The project sources data from the Alation Snowflake Data Share (read-only) and writes transformed classification data into the CLASSIFICATION, TAGGING, MASKING, and LOGGING schemas.

Model Layer Design (TBD)

Layer	Model	Materialisation	Target Schema	Description
Source	TBD	Source (external)	ALATION_SHARE	Read-only references to Alation Data Share tables
Staging	TBD	View	CLASSIFICATION	SELECT + validate + standardise from Alation share
Staging	TBD	View	CLASSIFICATION	Extract sensitivity level assignments
Classification	TBD	Incremental Table	CLASSIFICATION	Core model — final classification output with merge logic
Tagging	TBD	Incremental Table	TAGGING	Records of Snowflake tag assignments to apply
Masking	TBD	Incremental Table	MASKING	Records of masking policy assignments to apply
Logging	TBD	Incremental Table	LOGGING	Execution logs, row counts, error capture

dbt Project Configuration (dbt_project.yml)

The dbt_project.yml must define the following environment-specific targets: dev (developer Snowflake schema), syst (system integration), ppte (pre-production), and prod (production). Database and schema targets must be parameterised via environment variables to support CI/CD promotion without config changes.

Macro Design

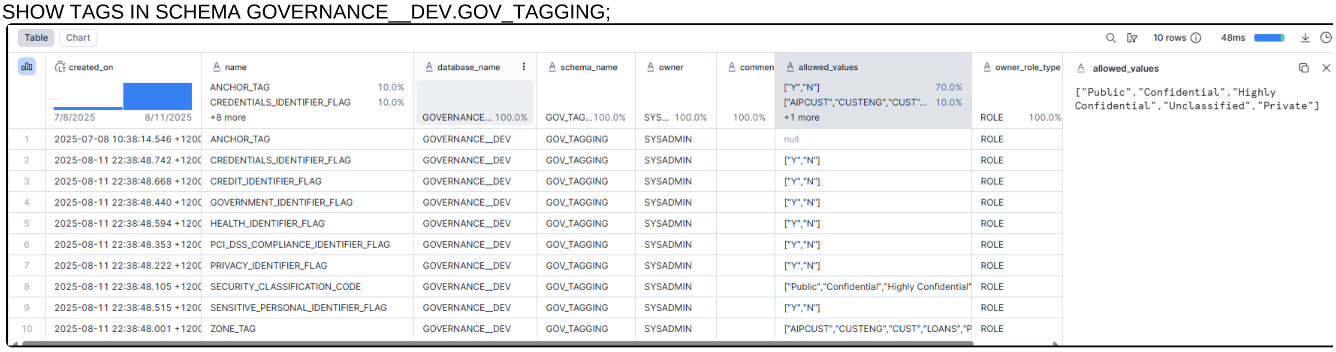
Macro 1: apply_classification_tags

Reads from CLASSIFICATION_DATA and generates ALTER TABLE / ALTER COLUMN statements to apply Snowflake tag objects. The macro iterates each row in the classification output and constructs the appropriate SQL.

It is idempotent — safe to re-run without creating duplicate tag assignments. Supports all classification types (PII, PCI, CONFIDENTIAL, PUBLIC).

Classification Tag

SHOW TAGS IN SCHEMA GOVERNANCE__DEV.GOV_TAGGING;



	created_on	name	database_name	schema_name	owner	commen	allowed_values	owner_role_type
	7/8/2025 8/11/2025	ANCHOR_TAG 10.0% CREDENTIALS_IDENTIFIER_FLAG 10.0% +8 more	GOVERNANCE... 100.0%	GOV_TAG... 100.0%	SYS... 100.0%	100.0%	["Y","N"] 70.0% ["AIPCUST","CUSTENG","CUST"... 10.0% +1 more	ROLE 100.0%
1	2025-07-08 10:38:14.546 +1200	ANCHOR_TAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		null	ROLE
2	2025-08-11 22:38:48.742 +1200	CREDENTIALS_IDENTIFIER_FLAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Y","N"]	ROLE
3	2025-08-11 22:38:48.668 +1200	CREDIT_IDENTIFIER_FLAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Y","N"]	ROLE
4	2025-08-11 22:38:48.440 +1200	GOVERNMENT_IDENTIFIER_FLAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Y","N"]	ROLE
5	2025-08-11 22:38:48.594 +1200	HEALTH_IDENTIFIER_FLAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Y","N"]	ROLE
6	2025-08-11 22:38:48.353 +1200	PCI_DSS_COMPLIANCE_IDENTIFIER_FLAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Y","N"]	ROLE
7	2025-08-11 22:38:48.222 +1200	PRIVACY_IDENTIFIER_FLAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Y","N"]	ROLE
8	2025-08-11 22:38:48.105 +1200	SECURITY_CLASSIFICATION_CODE	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Public","Confidential","Highly Confidential"	ROLE
9	2025-08-11 22:38:48.515 +1200	SENSITIVE_PERSONAL_IDENTIFIER_FLAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["Y","N"]	ROLE
10	2025-08-11 22:38:48.001 +1200	ZONE_TAG	GOVERNANCE__DEV	GOV_TAGGING	SYSADMIN		["AIPCUST","CUSTENG","CUST","LOANS","P	ROLE

ANCHOR_TAG
CREDENTIALS_IDENTIFIER_FLAG
CREDIT_IDENTIFIER_FLAG
GOVERNMENT_IDENTIFIER_FLAG
HEALTH_IDENTIFIER_FLAG
PCI_DSS_COMPLIANCE_IDENTIFIER_FLAG
PRIVACY_IDENTIFIER_FLAG
SECURITY_CLASSIFICATION_CODE
SENSITIVE_PERSONAL_IDENTIFIER_FLAG
ZONE_TAG

Classification	Tag Name	Tag Value
Highly Confidential	SECURITY_CLASSIFICATION_CODE	'Highly Confidential'
Confidential	SECURITY_CLASSIFICATION_CODE	'Confidential'
Private	SECURITY_CLASSIFICATION_CODE	'Private'
Public	SECURITY_CLASSIFICATION_CODE	'Public'
Unclassified	SECURITY_CLASSIFICATION_CODE	'Unclassified'

Macro 2: apply_masking_policies

Reads from MASKING_ASSIGNMENTS and generates the sequence of SQL statements required to apply Snowflake masking policies.

For each record, the macro first issues an UNSET MASKING POLICY statement to remove any existing policy before applying the new one, preventing policy conflicts. Designed to be safely re-executed in all environments.

Sensitivity Masking

SHOW MASKING POLICIES IN SCHEMA GOVERNANCE__DEV.GOV_MASKING;

Sensitivity	Data Type	Masking Policy
High	VARCHAR/TEXT/STRING	SENSITIVE_DATA_MASK_STRING

High	NUMBER/INT/DECIMAL	SENSITIVE_DATA_MASK_NUMBER
High	DATE	SENSITIVE_DATA_MASK_DATE
High	TIMESTAMP	SENSITIVE_DATA_MASK_TIMESTAMP
High	BOOLEAN	SENSITIVE_DATA_MASK_BOOLEAN
High (for keys)	Any	HASH_KEY
Medium	VARCHAR/TEXT/STRING	SENSITIVE_DATA_MASK_STRING
Medium	NUMBER/INT/DECIMAL	SENSITIVE_DATA_MASK_NUMBER
Medium (for keys)	Any	HASH_KEY

Macro 3: apply_object_tags

Reads tag assignments and applies object-level tags at both table and column granularity using ALTER TABLE SET TAG and ALTER TABLE ALTER COLUMN SET TAG syntax respectively. The macro generates the required SQL dynamically and is safe to run multiple times without causing errors or duplicate changes.

Hashing — BK / PK / FK Implementation

MASK_SHA256_HASH policy created apply on Key flag columns in CLASSIFICATION_DATA populated . BK/PK/FK tags provided by Data Governance.

Logging & Error Handling

- All macro executions write audit records to GOVERNANCE_DB.LOGGING.AUDIT_LOG, capturing: run timestamp, macro name, row count processed, success/failure status, and error message if applicable.
- dbt model run results are captured via dbt's built-in results object and persisted to the audit log table.
- Failed records are written to a LOGGING.ERROR_LOG table with the full record and error reason for investigation.
- Alerting on pipeline failures is triggered via Airflow task failure callbacks.

Incremental Processing

Strategy

- Use record_updated timestamp
- Hash-based change detection

Handling Changes

- New records Insert
- Updated Update
- Removed Optional delete/flag

Macro Responsibilities

- Read classification data
- Compare existing tags
- Apply delta changes
- Log actions

Source Setup & Alation Share Validation

Before staging model development, the following steps must be completed to ensure source integrity:

- Configure dbt sources to reference the Alation Data Share database in read-only mode.
- Confirm the dbt service account has SELECT privileges on all required Alation share tables.
- Profile all Alation share tables to validate structure, data types, and record counts against the approved STM.
- Validate that tag_name values in the share are restricted to the approved classification list: PII, PCI, CONFIDENTIAL, PUBLIC, SENSITIVITY_LEVEL.

DevOps Workflow & CI/CD

Branching Strategy

Branch	Purpose	Merge Target	PR Required
feature/*	Individual development tasks	develop	Yes — peer review

develop	Integration branch for testing	prod	Yes — tech lead approval
Prod	Production-ready code	—	Yes — governance sign-off

Orchestration — Airflow (Astro)

Pipeline DAG Design

The end-to-end classification pipeline will be orchestrated via Airflow running on the Astro platform. The Astro Airflow DAG that executes the full pipeline: Alation share validation, staging, classification, tagging, masking, and post-validation. Supports schedule and ad-hoc manual trigger. Passes time-based parameters to dbt at runtime.

Work flow

- Refresh metadata
- Run dbt staging
- Update classification table
- Apply tags
- Validate

Role-Based Access Control (RBAC)

RBAC Matrix

Role	Environment	HC Column (Highly Confidential)	CONF Column (Confidential)	PRIVATE Column	PUBLIC Column	Unclassified Column	TRANSFORM (Own Schema)	TRANSFORM (Other Dev)
DEVELOPER ROLES								
Landing Developer	DEV	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access
Subject Area Developer (HC Developer)	DEV	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access
TESTER / SUPPORT ROLES								
Subject Area Tester (HC Tier)	DEV, SYST, PPTE	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access
Subject Area Tester (CONFIDENTIAL Tier)	DEV, SYST, PPTE	MASKED	Plaintext	Plaintext	Plaintext	MASKED	Plaintext	No access
Subject Area Tester (PUBLIC / Private Tier)	DEV, SYST, PPTE	MASKED	MASKED	Plaintext	Plaintext	MASKED	Plaintext	No access
SUPPORT ROLES								
Subject Area Support (HC Tier)	DEV, SYST, PPTE, PROD	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access
Subject Area Support (CONFIDENTIAL Tier)	DEV, SYST, PPTE, PROD	MASKED	Plaintext	Plaintext	Plaintext	MASKED	Plaintext	No access
Subject Area Support (PUBLIC / Private Tier)	DEV, SYST, PPTE, PROD	MASKED	MASKED	Plaintext	Plaintext	MASKED	Plaintext	No access
ANALYST ROLES								
Subject Area Analyst (HC Tier)	PROD	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access	No access
Subject Area Analyst (CONFIDENTIAL Tier)	PROD	MASKED	Plaintext	Plaintext	Plaintext	MASKED	Plaintext	No access
Subject Area Analyst (PUBLIC / Private Tier)	PROD	MASKED	MASKED	Plaintext	Plaintext	MASKED	Plaintext	No access
SERVICE ACCOUNT ROLES								
Airflow Service Role	ALL	Plaintext	Plaintext	Plaintext	Plaintext	Plaintext	No access	No access

Testing

Test Scope

Test Type	Scope
dbt Schema Tests	All models — not null, unique, accepted values, referential integrity
Macro Unit Tests	Validate generated SQL for apply_classification_tags, apply_masking_policies, apply_object_tags
Integration Tests	Staging Classification model chain; data flows correctly end-to-end
Row Count Reconciliation	CLASSIFICATION_DATA row count vs Alation share source record count
Tag Verification	Confirm Snowflake tags applied correctly to all in-scope tables and columns
Masking Verification	Confirm masking policies applied; test role-based masking behaviour
Hashing Validation	Verify SHA-256 hashing applied correctly for PII fields
End-to-End Test	Full pipeline: Alation share staging classification tag application masking enforcement Governance DB

Conclusion

This solution provides a scalable, automated, and governance-driven framework leveraging Alation Data Share and Snowflake capabilities to enforce consistent data protection policies across the organization.